

Instituto Federal de Santa Catarina – Campus Caçador

Cálculo Numérico – Resolução da Prova com Códigos MATLAB

Aluno: Vitor Sales Dias da Rosa

Questão 1 Método do Ponto Fixo

Problema: $f(x) = \frac{x - 3\sqrt{x+3}}{x^2}$, raiz em $(11, 12)$, proposta $\varphi(x) = 3\sqrt{x+3}$, $x_0 = 1,0$.

Erro 1 — Ponto inicial fora da região: A raiz está em $(11, 12)$, mas $x_0 = 1,0$ é completamente diferente. O método do ponto fixo exige que x_0 esteja na vizinhança da raiz.

Erro 2 — Verificar convergência antes de aceitar φ : A derivada de $\varphi(x) = 3\sqrt{x+3}$ é

$$\varphi'(x) = \frac{3}{2\sqrt{x+3}}$$

Em $x^* \approx 11,6$:

$$|\varphi'(11,6)| = \frac{3}{2\sqrt{14,6}} \approx \frac{3}{7,64} \approx 0,39 < 1 \Rightarrow \text{converge na vizinhança certa}$$

Portanto $\varphi(x)$ está **algebricamente correta**, mas é necessário $x_0 \in (11, 12)$.

$$\varphi(x) = 3\sqrt{x+3}, \quad x_0 = 11,5$$

Primeiras iterações:

k	x_k	$ x_{k+1} - x_k $
0	11,5000	—
1	$3\sqrt{14,5} = 11,431$	0,069
2	$3\sqrt{14,431} = 11,402$	0,029
...	...	...

Converge para $x^* \approx 11,38$ onde $f(x^*) = 0$. $|\varphi'(x^*)| < 1 \checkmark$

```

1 % phi define a funcao de iteracao x = phi(x)
2 % Tres opcoes foram testadas (duas comentadas):
3 % phi = @(x) x^2 - 2           % Nao converge (|phi'| > 1 na raiz)
4 % phi = @(x) sqrt(x+2)        % Converte para sqrt(2)
5 phi = @(x) (x + 2) / x        % Converte para raiz de x^2 - 2 = 0
6
7 x1 = 5;           % Chute inicial (deve estar perto da raiz)

```

```

8 error = 1;      % Inicializa o erro acima da tolerancia
9
10 while(error > 0.000001) % Critério de parada: |x2 - x1| < tol
11     x2 = phi(x1);      % Aplica a iteracao
12     error = abs(x2-x1); % Calcula o erro absoluto
13     x1 = x2;           % Avanca para o proximo passo
14 end
15
16 x1 % Raiz aproximada
17 x2 % Mesmo valor (ultima iteracao)

```

Funcionamento: a cada iteração aplica $x \leftarrow \varphi(x)$ e mede quanto x mudou. Para à medida que $|x_{k+1} - x_k|$ fica menor que 10^{-6} .

Condição de convergência verificada no código: basta comparar as três funções comentadas – $\varphi(x) = x^2 - 2$ diverge porque $|\varphi'(x^*)| = |2x^*| = 2\sqrt{2} > 1$. As outras duas têm $|\varphi'| < 1$ e convergem.

Questão 2 Método da Bissecção

Problema: $f(x) = x^8 - 2$, $[a, b] = [1; 1,1]$, $\varepsilon = 0,1$.

O enunciado afirma que $|f(x_5)| < \varepsilon = 0,1$ com $x_5 = 1,071875$.

Calculando: $f(1,071875) = (1,071875)^8 - 2 \approx 1,743 - 2 = -0,257$

$\Rightarrow |f(x_5)| \approx 0,257 > 0,1$ **FALSO** – o método não podia parar.

O critério adequado é $\frac{b-a}{2} < \varepsilon$. As iterações aritméticas estão corretas; o problema é apenas o critério de parada declarado.

Já em $x_1 = 1,05$: $\frac{1,1 - 1,0}{2} = 0,05 < 0,1 \checkmark$ Logo o método poderia parar em x_1 com esse critério.

Questão 3 Eliminação de Gauss

Resolver:
$$\begin{bmatrix} 6 & 2 & -1 \\ 2 & 4 & 1 \\ 3 & 2 & 8 \end{bmatrix} x = \begin{bmatrix} 7 \\ 7 \\ 13 \end{bmatrix}$$

Fase 1 – Triangularização

Matriz aumentada: $\left[\begin{array}{ccc|c} 6 & 2 & -1 & 7 \\ 2 & 4 & 1 & 7 \\ 3 & 2 & 8 & 13 \end{array} \right]$

$$L_2 \leftarrow L_2 - \frac{2}{6}L_1: \left[\begin{array}{ccc|c} 6 & 2 & -1 & 7 \\ 0 & 10/3 & 4/3 & 14/3 \\ 3 & 2 & 8 & 13 \end{array} \right]$$

$$L_3 \leftarrow L_3 - \frac{3}{6}L_1: \left[\begin{array}{ccc|c} 6 & 2 & -1 & 7 \\ 0 & 10/3 & 4/3 & 14/3 \\ 0 & 1 & 17/2 & 19/2 \end{array} \right]$$

$$L_3 \leftarrow L_3 - \frac{3}{10}L_2: \left[\begin{array}{ccc|c} 6 & 2 & -1 & 7 \\ 0 & 10/3 & 4/3 & 14/3 \\ 0 & 0 & 81/10 & 81/10 \end{array} \right]$$

Fase 2 – Substituição regressiva

$$L_3: \frac{81}{10}x_3 = \frac{81}{10} \Rightarrow \boxed{x_3 = 1} \quad L_2: \frac{10}{3}x_2 + \frac{4}{3} = \frac{14}{3} \Rightarrow \boxed{x_2 = 1} \quad L_1: 6x_1 + 2 - 1 = 7 \Rightarrow \boxed{x_1 = 1}$$

$$\therefore x = [1 \ 1 \ 1]^\top \checkmark$$

```

1  A = [2 -3 1; 4 -6 -1; 1 2 1];
2  B = [-5; -7; 4];
3  M = [A B]; % Monta a matriz aumentada [A/b]
4  [l c] = size(A); % l = linhas, c = colunas de A
5
6  %% FASE 1: Triangularizacao (zerar abaixo da diagonal)
7  for j = 1:c-1 % Para cada coluna pivot j
8
9      % Pivotacao parcial: troca a linha atual pela de maior valor
      absoluto
10     maior = max(abs(M(j:end, j)));
11     LinhaPermuta = j-1 + find(M(j:end,j) == maior);
12     aux = M(j,:); M(j,:) = M(LinhaPermuta,:); M(LinhaPermuta,:) =
        aux;
13
14     for i = j+1:l % Para cada linha abaixo do pivot
15         a = M(i,j) / M(j,j); % Multiplicador
16         M(i,:) = M(i,:) - a * M(j,:); % Eliminacao
17     end
18 end
19
20 %% FASE 2: Normalizar diagonal (cada linha / seu pivot)
21 for i = 1:l
22     M(i,:) = M(i,:) / M(i,i); % Linha i dividida pelo pivot
23 end
24
25 %% FASE 3: Substituicao regressiva (zerar acima da diagonal)
26 for i = l-1:-1:1 % De baixo para cima
27     s = 0;
28     for j = l:-1:i+1 % Soma os termos ja conhecidos
29         s = s + M(j, c+1) * M(i,j);
30     end
31     M(i, c+1) = M(i, c+1) - s; % Atualiza o termo independente
32 end
33

```

```
34 | solucao = M(:, c+1)      % Ultima coluna = vetor solucao x
```

Detalhe importante: este código implementa **Gauss-Jordan com pivotação parcial**. A pivotação (troca de linhas) evita divisão por zero e melhora a estabilidade numérica. O resultado final é a forma diagonal ($M = I$), e a solução está na última coluna.

Questão 4 Método de Newton (N dimensões)

Interseção de $x^2 + y^2 = 4$ e $y = 3 \sin(\frac{3}{2}x)$, $X^{(0)} = (0, 1)^\top$.

Formulação: $F(x)$

$$F(x, y) = \begin{bmatrix} x^2 + y^2 - 4 \\ y - 3 \sin(\frac{3}{2}x) \end{bmatrix}, \quad J(x, y) = \begin{bmatrix} 2x & 2y \\ -\frac{9}{2} \cos(\frac{3}{2}x) & 1 \end{bmatrix}$$

Iteração: $X^{(k+1)} = X^{(k)} - J(X^{(k)})^{-1} F(X^{(k)})$

$1)^\top$

Iteração 1 com $X^{(0)}$

$$F(0, 1) = \begin{bmatrix} 0 + 1 - 4 \\ 1 - 0 \end{bmatrix} = \begin{bmatrix} -3 \\ 1 \end{bmatrix}, \quad J(0, 1) = \begin{bmatrix} 0 & 2 \\ -4,5 & 1 \end{bmatrix}$$

$$\det J = 0 \cdot 1 - 2 \cdot (-4,5) = 9$$

$$J^{-1} = \frac{1}{9} \begin{bmatrix} 1 & -2 \\ 4,5 & 0 \end{bmatrix}$$

$$\Delta X = -J^{-1}F = -\frac{1}{9} \begin{bmatrix} -3 - 2 \\ -13,5 \end{bmatrix} = -\frac{1}{9} \begin{bmatrix} -5 \\ -13,5 \end{bmatrix} = \begin{bmatrix} 0,556 \\ 1,5 \end{bmatrix}$$

$$X^{(1)} = \begin{bmatrix} 0,556 \\ 2,5 \end{bmatrix}$$

O método converge em ≈ 4 –5 iterações para a raiz próxima a $(0, 1)$.

```
1 % Define o sistema F(x,y) = 0
2 % F1: circulo x^2 + y^2 - 1 = 0
3 % F2: parabola y - x^2 = 0
4 F = @(x,y) [x^2 + y^2 - 1; y - x^2];
5
6 % Jacobiana J: matriz das derivadas parciais
7 % J(1,1) = dF1/dx = 2x    J(1,2) = dF1/dy = 2y
8 % J(2,1) = dF2/dx = -2x   J(2,2) = dF2/dy = 1
9 J = @(x,y) [2*x, 2*y;
10             -2*x, 1];
11
```

```

12 X0 = [1; 0];      % Condição inicial (perto de uma das interseções)
13
14 x = X0(1);  y = X0(2);
15
16 % Primeira iteração:  $X = X_0 - J^{-1} * F(X_0)$ 
17 X = X0 - inv(J(x,y)) * F(x,y);
18
19 % Critério de parada: soma dos erros absolutos em cada componente
20 error = sum(abs(X - X0));
21
22 while (error > 0.001)
23     X0 = X;
24     x = X0(1);  y = X0(2);
25
26     X = X0 - inv(J(x,y)) * F(x,y);  % Iteração de Newton N-D
27
28     error = sum(abs(X - X0));  % Atualiza erro
29 end
30
31 X  % Raiz aproximada: ponto de interseção das duas curvas

```

Pontos-chave do código:

- $\text{inv}(J)*F$ calcula $J^{-1}F$ (em problemas grandes prefere-se $J \backslash F$ por ser mais estável)
- O critério de parada usa a soma das variações em todas as componentes
- A Jacobiana é reavaliada a cada iteração (Newton clássico)
- O código resolve $x^2 + y^2 = 1$ e $y = x^2$; para a questão da prova basta trocar F e J

Adaptação para a prova ($x^2 + y^2 = 4$ e $y = 3\sin(1,5x)$):

```

1 F = @(x,y) [x^2 + y^2 - 4;  y - 3*sin(1.5*x)];
2 J = @(x,y) [2*x,          2*y;
3             -4.5*cos(1.5*x), 1];
4 X0 = [0; 1];

```

Questão 5 Jacobi-Richardson vs. Gauss-Seidel

O código da prova faz $\mathbf{x}_2 = -\mathbf{LR}*\mathbf{x}_1 + \mathbf{b}$, mas a fórmula correta é:

$$x^{(k+1)} = D^{-1}(b - L_R x^{(k)})$$

Falta dividir pelo vetor diagonal D . O sistema 1 “funcionava” porque sua diagonal é toda 1 ($D = I$). O sistema 2 tem diagonal $[4, 3, 8, 2]$ e por isso divergia. A correção é substituir $\mathbf{x}_2 = -\mathbf{LR}*\mathbf{x}_1 + \mathbf{b}$ por $\mathbf{x}_2 = D \backslash (\mathbf{b} - \mathbf{LR}*\mathbf{x}_1)$.

```

1 A = [10 2 1; 1 5 1; 2 3 10];
2 b = [7; -8; 6];
3 [m n] = size(A);
4 x0 = [0; 0; 0];      % Chute inicial
5
6 % Pre-processamento: divide cada equacao pelo seu coeficiente diagonal
7 % Isto transforma A(i,i) em 1 e incorpora a divisao por D
8 for i = 1:m
9     b(i,:) = b(i,:) / A(i,i);    % Normaliza o termo independente
10    A(i,:) = A(i,:) / A(i,i);    % Normaliza a linha i
11    A(i,i) = 0;                  % Zera a diagonal (formando LR = L+U)
12 end
13 % Apos o loop: A agora e' LR = D^{-1}(L+U), b = D^{-1}*b_original
14 % A formula vira: x^{k+1} = b - LR * x^{k} (divisao ja foi feita!)
15
16 x1 = b - A * x0;                % Primeira iteracao
17 erro = max(abs(x1 - x0));
18
19 while (erro > 0.01)
20     x0 = x1;
21     x1 = b - A * x0;    % JACOBI: usa x0 (iteracao anterior) inteiro
22     % DIFERENCA do Gauss-Seidel: aqui x0 nao e'
23     % atualizado
24     % durante o loop -- todos usam o mesmo x^{(k)}
25     erro = max(abs(x1 - x0));
26 end
27 A      % Matriz LR normalizada (diagonal = 0)
28 b      % Vetor b normalizado
29 x1     % Solucao aproximada

```

Resumo da diferença Jacobi × Gauss-Seidel:

	Jacobi	Gauss-Seidel
Usa	$x^{(k)}$ completo para calcular $x^{(k+1)}$	usa $x_j^{(k+1)}$ assim que disponível
Loop interno	sem loop (operação matricial)	loop for i=1:n
MATLAB	$x1 = b - A*x0$	$x1(i) = b(i) - A(i,:)*x1$
Convergência	mais lenta	geralmente mais rápida
Paralelizável	sim	não

```

1 A = [10 2 1; 1 5 1; 2 3 10];
2 b = [7; -8; 6];
3 [m n] = size(A);
4 x0 = [0; 0; 0];
5
6 % Mesmo pre-processamento do Jacobi
7 for i = 1:m
8     b(i,:) = b(i,:) / A(i,i);
9     A(i,:) = A(i,:) / A(i,i);
10    A(i,i) = 0;
11 end

```

```

12 x1 = b - A * x0;
13 erro = max(abs(x1 - x0));
14
15 while (erro > 0.0001)
16     x0 = x1;
17
18     for i = 1:n
19         % GAUSS-SEIDEL: ao calcular x1(i), usa os valores JA
20         % ATUALIZADOS
21         % de x1(1..i-1) (calculados nesta mesma iteracao)
22         % e os valores antigos x1(i+1..n) (ainda do passo anterior)
23         x1(i) = b(i) - A(i,:) * x1; % x1 ja tem novos valores para j
24         % < i
25     end
26
27     erro = max(abs(x1 - x0));
28 end
29 x1 % Solucao aproximada

```

Bónus – Série de Taylor (taylor_serie.m)

```

1 max = 10; % Numero de termos (i = 0, 1, ..., 10 => 11 termos)
2 s = 0; % Acumulador da soma
3 x = 0.1; % Ponto onde se avalia e^x
4
5 % Expansao de Taylor de e^x em torno de 0:
6 % e^x = sum_{i=0}^{inf} x^i / i!
7 for i = 0:max
8     s = s + x^i / factorial(i); % Adiciona o i-esimo termo
9 end
10
11 exp(x) % Valor exato (MATLAB) ~ 1.10517
12 s % Aproximacao por Taylor com 11 termos

```

Com $x = 0,1$ e 11 termos: $s \approx 1,10517$, igual a $\exp(0.1)$ até à precisão da máquina – a série converge muito rápido para $|x| < 1$.

Fim da resolução completa